

QueryMind AI

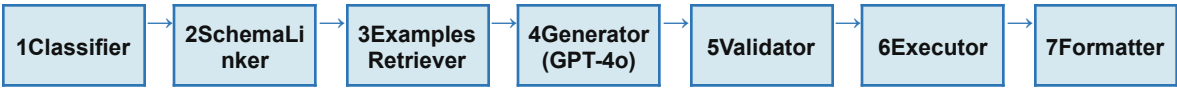
Complete Lifecycle Walkthrough

Step-by-step guide to every component — how it works, how to test it, and how it connects to the next

Version	1.0
Audience	Developers, QA, Demo team
Stack	FastAPI · PostgreSQL · Qdrant · Docling · OpenAI GPT-4o · Next.js
Sections	10 lifecycle steps + Appendix

How the Full Pipeline Works

When a user submits a natural language question, QueryMind runs it through 7 sequential steps. Every component you configure in the admin panels directly affects the output of this pipeline.



Step 1 — Classifier	Detects intent: analytical query, invalid request, or ambiguous input
Step 2 — Schema Linker	Maps words in the question to actual table/column names in PostgreSQL
Step 3 — Examples Retriever	Semantic search in Qdrant to find the most similar past Q→SQL pairs
Step 4 — Generator (GPT-4o)	Sends schema + few-shot examples + question to OpenAI, gets back SQL
Step 5 — Validator	Checks guardrails (SELECT only?), SQL syntax, and row limits
Step 6 — Executor	Runs validated SQL against PostgreSQL, enforces max_rows cap
Step 7 — Formatter	Returns result table + a natural-language summary of the answer

Step 1 — Start Infrastructure

What it does

Starts two services in Docker: PostgreSQL (the analytical database) and Qdrant (the vector database for semantic search). Nothing else in the application works until both are healthy.

Services at a glance

PostgreSQL 16	Port 5432 Stores e-commerce data + all admin config tables
Qdrant	Port 6333 Stores vector embeddings for few-shot examples and ingested documents

Steps

1. Open Docker Desktop and confirm it is running
2. From the project root, run:

```
docker compose up -d
```

3. Verify Postgres is healthy:

```
docker compose ps
```

Expected: STATUS column shows (healthy) for postgres

4. Verify Qdrant is healthy:

```
curl http://localhost:6333/healthz
```

Expected response: healthz check passed

5. Open Qdrant dashboard to confirm UI loads:

```
http://localhost:6333/dashboard
```

WHY

Qdrant persists vectors to the qdrant_data Docker volume — data survives restarts. Postgres does the same via postgres_data. You only need to seed data once.

Step 2 — Seed the Database

What it does

Populates PostgreSQL with realistic synthetic e-commerce data and the admin config tables that QueryMind needs to function. This is a one-time operation.

Data created

customers	1,000 rows name, email, country, segment (standard / premium / enterprise)
products	500 rows name, SKU, category, supplier, price, stock
categories	7 rows Electronics, Clothing, Home & Garden, Sports, Books, Toys, Food & Beverage
suppliers	20 rows name, country, email
orders	3,000 rows customer, status (pending/shipped/delivered/cancelled), total, dates
order_items	~9,000 rows order, product, quantity, unit_price, line_total (computed)
reviews	2,000 rows product, customer, rating 1-5, body text
guardrail_config	6 rows allow_select=true, allow_insert/update/delete=false, max_rows=500
model_config	4 rows model=gpt-4o, temperature=0.0, max_tokens=1000, dialect=postgresql

Steps

6. Make sure the backend virtual environment is active:

```
cd backend
```

7. Run the seed script:

```
uv run python ../scripts/seed_db.py
```

Expected output:

```
Seed complete.
Suppliers: 20 | Categories: 7 | Products: 500
Customers: 1000 | Orders: 3000 | Reviews: 2000
```

8. Verify in DBeaver — connect with these credentials:

Host	localhost
Port	5432

Database	querymind
Username	querymind
Password	querymind

9. Run a quick count in DBeaver to confirm:

```
SELECT 'customers' AS tbl, COUNT(*) FROM customers
UNION ALL SELECT 'orders', COUNT(*) FROM orders
UNION ALL SELECT 'products', COUNT(*) FROM products;
```

NOTE

If you wipe and restart Postgres, re-run seed_db.py. Qdrant data (embeddings) is independent — managed separately through the Examples Manager.

Step 3 — Start Backend and Frontend

What it does

Starts the FastAPI backend (the brains of the system) and the Next.js frontend (the UI). The backend creates any missing admin tables on startup, connects to Postgres and Qdrant, and exposes the REST API.

Environment setup

10. Copy the example env file if not already done:

```
cp backend/.env.example backend/.env
```

11. Open backend/.env and set your OpenAI key:

```
OPENAI_API_KEY=sk-...your-real-key-here...
```

Start Backend — Terminal 1

```
cd backend
uv run uvicorn app.main:app --reload --port 8000
```

Verify:

```
curl http://localhost:8000/health
```

Expected: {"status":"ok","service":"QueryMind AI"}

API docs available at: <http://localhost:8000/docs>

Start Frontend — Terminal 2

```
cd frontend
npm run dev
```

Verify: <http://localhost:3000> loads in browser — sidebar with navigation visible.

WHY

The backend's lifespan handler runs `Base.metadata.create_all` on startup — this is what creates the admin tables (`few_shot_examples`, `guardrail_config`, etc.) the first time. Always start the backend before seeding.

Step 4 — Schema Manager

What it does

Lets you add human-readable descriptions and synonyms to database columns. The Schema Linker (Step 2 of the query pipeline) uses these descriptions to map words in the user's question to the correct table and column names.

Why it matters

Without descriptions, the LLM must guess what columns mean from their names alone. With descriptions, it understands that:

- "segment" column means customer tier: standard, premium, or enterprise
- "status" on orders means lifecycle state: pending, shipped, delivered, cancelled
- "line_total" is the computed revenue per order line item

How it works

Storage	PostgreSQL — schema_columns table
API	GET / POST / PATCH / DELETE /admin/schema/columns
Pipeline	Schema Linker reads these at query time to build a context-rich schema prompt for the LLM

Steps — via API (<http://localhost:8000/docs>)

12. Introspect live schema — see all tables and columns:

```
GET /admin/schema/introspect
```

13. Add a description for the orders.status column:

```
POST /admin/schema/columns
{
  "table_name": "orders",
  "column_name": "status",
  "description": "Order lifecycle state. Values: pending, shipped, delivered, cancelled",
  "synonyms": ["state", "order state", "lifecycle", "fulfillment status"],
  "is_sensitive": false
}
```

14. Add description for customers.segment:

```
POST /admin/schema/columns
{
  "table_name": "customers",
  "column_name": "segment",

```



```
"description": "Customer tier. Values: standard, premium, enterprise",  
"synonyms": ["tier", "customer type", "plan", "account type"],  
"is_sensitive": false  
}
```

15. Recommended columns to describe for best query quality:

orders.status	pending / shipped / delivered / cancelled
customers.segment	standard / premium / enterprise
order_items.line_total	Computed revenue per line item (quantity x unit_price)
products.is_active	Whether the product is currently available for sale
products.stock_quantity	Current units available in inventory
orders.discount_amount	Discount applied to the order total

16. Verify: GET /admin/schema/columns returns all added records

TIP

Add synonyms generously. A user might say 'tier' when the column is called 'segment', or 'revenue' when the column is 'line_total'. Synonyms bridge that gap.

Step 5 — Examples Manager

What it does

Manages few-shot examples — pairs of a natural language question and the correct SQL. When a user asks a question, the system finds the most semantically similar examples and includes them in the LLM prompt. This dramatically improves SQL accuracy.

How the data flows

PostgreSQL	few_shot_examples table — source of truth (question, sql, query_type)
Qdrant	few_shot_examples collection — only the question is embedded and stored as a vector; SQL travels as payload metadata
At query time	User question is embedded → vector search in Qdrant → top-4 similar examples retrieved → injected into GPT-4o prompt

Why semantic search matters

A keyword search would miss this match:

```
Stored example: "What were the top 5 products by revenue?"
User question: "Best selling items this month"
```

These say different things but mean the same thing. Qdrant's cosine similarity on the embeddings catches this, so the right SQL pattern is always retrieved.

Steps — Add your first examples

17. Create example 1 — simple count:

```
POST /admin/examples
{
  "question": "How many customers do we have?",
  "sql": "SELECT COUNT(*) AS total_customers FROM customers",
  "query_type": "analytical"
}
```

18. Create example 2 — top-N with JOIN:

```
POST /admin/examples
{
  "question": "What are the top 5 products by revenue?",
  "sql": "SELECT p.name, SUM(oi.line_total) AS revenue FROM order_items oi JOIN products p ON p.id = oi.product_id GROUP BY p.name ORDER BY revenue DESC LIMIT 5",
  "query_type": "analytical"
}
```

19. Create example 3 — date filter:

```
POST /admin/examples
{
  "question": "How many orders were placed last month?",
  "sql": "SELECT COUNT(*) FROM orders WHERE created_at >= NOW() - INTERVAL '1 month'",
  "query_type": "analytical"
}
```

20. Create example 4 — segment filter:

```
POST /admin/examples
{
  "question": "Show me total revenue by customer segment",
  "sql": "SELECT c.segment, SUM(o.total_amount) AS revenue FROM orders o JOIN customers c ON c.id = o.customer_id GROUP BY c.segment ORDER BY revenue DESC",
  "query_type": "analytical"
}
```

21. Verify in Qdrant dashboard (<http://localhost:6333/dashboard>):

- Open Collections → few_shot_examples
- Points count should equal number of examples you added
- Click any point to see the payload (question, sql, query_type)

22. Sync embeddings (use after bulk imports):

```
POST /admin/examples/sync-embeddings
```

Expected: {"message": "Sync complete"}

TIP

Aim for 20-30 examples covering your most common query patterns: counts, top-N, date ranges, JOINS, GROUP BY, and segment filters. Quality of examples = quality of generated SQL.

Step 6 — Guardrails Config

What it does

Guardrails act as a safety layer in Step 5 of the query pipeline (Validator). They control exactly what the generated SQL is allowed to do — preventing destructive operations, limiting result size, and restricting schema access.

Default guardrail values

allow_select	true — SELECT queries are allowed (read-only analytics)
allow_insert	false — INSERT is blocked
allow_update	false — UPDATE is blocked
allow_delete	false — DELETE is blocked
max_rows	500 — Result sets are capped at 500 rows
restrict_to_schema	public — Queries are restricted to the public schema

Steps

23. View all guardrails:

```
GET /admin/guardrails
```

- 24. Verify allow_select = true, all others = false
- 25. Test the max_rows limit — temporarily lower it:

```
PATCH /admin/guardrails/max_rows{"value": "10"}
```

26. Run this query: POST /query with {"question": "List all customers"}

Result set should contain exactly 10 rows even though 1,000 exist

27. Restore max_rows:

```
PATCH /admin/guardrails/max_rows{"value": "500"}
```

28. Test INSERT blocking — run this query:

```
POST /query{"question": "Add a new customer named John"}
```

Expected: error response — INSERT not allowed by guardrails

WHY

The Validator step checks the generated SQL against these flags before execution. If allow_delete=false and the LLM somehow generates DELETE, it is blocked here — the SQL never reaches the database.

Step 7 — Model Config

What it does

Controls the LLM settings used by the Generator step (Step 4 of the pipeline). Changes take effect immediately on the next query — no restart required.

Config keys

model	OpenAI model to use. Default: gpt-4o. Can be changed to gpt-4o-mini for lower cost
temperature	0.0 = deterministic (same input = same SQL). Higher = more creative but less reliable. Keep at 0.0 for SQL generation
max_tokens	Maximum tokens in the LLM response. 1000 is sufficient for most SQL. Increase for very complex queries
dialect	SQL dialect hint sent to the LLM. Default: postgresql

Steps

29. View current config:

```
GET /admin/config
```

30. Update temperature for testing (then restore):

```
PATCH /admin/config/temperature{"value": "0.1"}
```

31. Switch to a cheaper model for demo (optional):

```
PATCH /admin/config/model{"value": "gpt-4o-mini"}
```

32. Restore defaults after testing:

```
PATCH /admin/config/temperature {"value": "0.0"}
PATCH /admin/config/model {"value": "gpt-4o"}
```

TIP

Keep temperature=0.0 for production. SQL generation is not a creative task — consistency and correctness matter far more than variety.

Step 8 — Document Ingestion (Docling + Qdrant)

What it does

Lets you upload schema documentation, data dictionaries, or any reference documents. Docling parses them into structured text, which is chunked and stored in a separate Qdrant collection called schema_docs. These chunks can be semantically searched for additional context.

How the ingestion pipeline works

1. Upload	You POST a file to /admin/documents/upload
2. Docling	Converts PDF/DOCX/HTML/MD to structured markdown — handles tables, headings, multi-column layouts
3. Chunking	Markdown is split into ~400-word overlapping chunks
4. Embed	Each chunk is embedded with OpenAI text-embedding-3-small (1536 dimensions)
5. Qdrant	Chunks stored in schema_docs collection with payload: {text, doc_name, chunk_index}

Two Qdrant collections — side by side

few_shot_examples	Question→SQL pairs for few-shot prompting. Indexed by Examples Manager.
schema_docs	Document chunks from uploaded files. Indexed by Document Ingestion.

Supported formats

- .pdf — data dictionaries, schema docs, reports
- .docx — Word documents
- .html / .htm — web pages, exported reports
- .md — Markdown files (e.g. README, wiki pages)
- .txt — plain text

Steps

33. Prepare a document — create a simple text file to test:

```
# Schema Reference
## orders table
The orders table tracks customer purchases.
Status values: pending, shipped, delivered, cancelled.
discount_amount is the total discount applied to the order.
```

Save it as schema_reference.md

34. Upload via API:

```
POST /admin/documents/upload
Content-Type: multipart/form-data
file: schema_reference.md
```

Expected: {"doc_name": "schema_reference.md", "chunks_indexed": 1}

35. Verify in Qdrant dashboard:

- Open Collections → schema_docs
- Points count = number of chunks indexed
- Click a point to inspect payload: {text, doc_name, chunk_index}

36. Search the indexed content:

```
GET /admin/documents/search?q=order+status+values&top_k=3
```

Expected: top-3 relevant chunks with text, doc_name, and similarity score

37. Test unsupported file type:

```
POST /admin/documents/upload with data.csv
```

Expected: HTTP 422 Unsupported file type .csv

38. Delete a document:

```
DELETE /admin/documents/schema_reference.md
```

Expected: HTTP 204. Qdrant points removed.

WHY

Upload your actual schema documentation or data dictionary. The richer the context in schema_docs, the better the LLM understands edge cases like computed columns, business rules, and valid enum values.

Step 9 — Run a Query (Full Pipeline Walkthrough)

What it does

This is the core feature of QueryMind. You send a natural language question and receive a SQL result set plus a plain-English summary. All 7 pipeline steps execute in sequence.

API

```
POST http://localhost:8000/query
Body: {"question": "your question here"}
```

What happens inside — step by step

Step 1 Classifier	Your question is sent to GPT-4o. It classifies it as analytical, invalid, or ambiguous. If invalid/ambiguous, the pipeline stops here with a message.
Step 2 Schema Linker	Extracts entities from your question (e.g. 'customers', 'segment'). Looks them up against schema_columns descriptions and synonyms to build a focused schema context.
Step 3 Retriever	Your question is embedded with text-embedding-3-small. Qdrant searches few_shot_examples for the top-4 most similar questions. Their SQL patterns become examples.
Step 4 Generator	GPT-4o receives: the schema context + 4 few-shot examples + your question. It returns SQL.
Step 5 Validator	Checks: Is it SELECT only (guardrail)? Is the syntax valid (sqlparse)? Would it exceed max_rows?
Step 6 Executor	Runs the SQL against PostgreSQL. Result is capped at max_rows. Latency and row count are recorded.
Step 7 Formatter	Structures the rows as a table. GPT-4o writes a 1-2 sentence natural-language summary of the result.

Test queries — run each one

39. Simple count:

```
POST /query {"question": "How many customers do we have?"}
```

40. Top-N with aggregation:

```
POST /query {"question": "What are the top 5 products by revenue?"}
```

41. Date range filter:

```
POST /query {"question": "How many orders were placed last month?"}
```

42. Multi-table JOIN:

```
POST /query {"question": "Show me average order value by customer segment"}
```

43. Trend analysis:

```
POST /query {"question": "What is the monthly revenue trend for the past year?"}
```

44. Inventory query:

```
POST /query {"question": "Which suppliers have the most out-of-stock products?"}
```

45. Rating analysis:

```
POST /query {"question": "What is the average product rating by category?"}
```

Edge case queries

46. Ambiguous (no specific intent):

```
POST /query {"question": "Show me data"}
```

Expected: classified as ambiguous, error or clarification returned, no SQL generated

47. Off-topic:

```
POST /query {"question": "What is the capital of France?"}
```

Expected: classified as invalid, rejected before SQL generation

48. No results:

```
POST /query {"question": "List all orders placed in 1990"}
```

Expected: success, empty result set, row_count=0

49. SQL injection attempt:

```
POST /query {"question": "Show all users; DROP TABLE customers;--"}
```

Expected: Validator blocks it, status=error, customers table intact (verify in DBeaver)

Step 10 — Query Logs

What it does

Every query run through the pipeline is automatically logged to the query_logs PostgreSQL table. The logs give full observability into what SQL was generated, how long it took, how many rows were returned, and whether it succeeded or failed.

Log fields

nl_query	The original natural language question
query_type	Classification result: analytical / invalid / ambiguous
generated_sql	The SQL the LLM generated (or null if pipeline stopped early)
status	success or error
error_message	Reason for failure (null if successful)
latency_ms	Total pipeline time in milliseconds
row_count	Number of rows returned from PostgreSQL
created_at	Timestamp of the query

Steps

50. Run several queries via POST /query first

51. List all logs (paginated):

```
GET /admin/logs?page=1&page_size=20
```

52. Filter by success only:

```
GET /admin/logs?status=success
```

53. Filter by error only:

```
GET /admin/logs?status=error
```

54. Check latency — run this SQL directly in DBeaver:

```
SELECT nl_query, status, latency_ms, row_count, created_at
FROM query_logs
ORDER BY created_at DESC
LIMIT 20;
```

55. Find the slowest queries:

```
SELECT nl_query, latency_ms, generated_sql
FROM query_logs
ORDER BY latency_ms DESC
LIMIT 5;
```

TIP

Query logs are your feedback loop. If a query has status=error, inspect generated_sql to see what the LLM produced — then add a corrected example to the Examples Manager to prevent it happening again.

Appendix — Quick Reference

Service URLs

Frontend App	http://localhost:3000
API Docs (Swagger)	http://localhost:8000/docs
Backend Health	http://localhost:8000/health
Qdrant Dashboard	http://localhost:6333/dashboard
PostgreSQL	localhost:5432 / querymind / querymind

Complete API Reference

POST /query	Submit a natural language question → returns SQL result + summary
GET /admin/schema/introspect	Live schema map from PostgreSQL
GET /admin/schema/columns	List all column descriptions
POST /admin/schema/columns	Add a column description
PATCH /admin/schema/columns/{id}	Update a column description
DELETE /admin/schema/columns/{id}	Delete a column description
GET /admin/examples	List all few-shot examples
POST /admin/examples	Add a new example (auto-indexed in Qdrant)
PATCH /admin/examples/{id}	Update an example (re-indexed in Qdrant)
DELETE /admin/examples/{id}	Delete an example (removed from Qdrant)
POST /admin/examples/sync-embeddings	Re-index all examples into Qdrant
GET /admin/guardrails	List all guardrail settings
PATCH /admin/guardrails/{key}	Update a guardrail value
GET /admin/config	List model config settings
PATCH /admin/config/{key}	Update a model config value
GET /admin/logs	Query log with pagination and status filter

POST /admin/documents/upload	Upload and index a document (PDF/DOCX/HTML/MD/TXT)
DELETE /admin/documents/{doc_name} }	Remove a document's chunks from Qdrant
GET /admin/documents/search	Semantic search over ingested document chunks

Stop all services

```
# Kill backend (port 8000) and frontend (port 3000)
kill $(lsof -ti :8000) 2>/dev/null
kill $(lsof -ti :3000) 2>/dev/null

# Stop Docker containers
docker compose down
```

Restart (data persists)

```
docker compose up -d
cd backend && uv run uvicorn app.main:app --reload --port 8000
cd frontend && npm run dev
```